



Database Management Systems

SQL overview, Data definition,
Structure of SQL queries

Anchor: Dr. Nagasundari S, Dept of CSE

Database Management Systems

Unit 1: Introduction to Database Management and SQL

Slides adapted from Author Slides of “Database System Concepts”, Silberschatz, H Korth and S Sudarshan, McGrawHill, 7th Edition, 2019. And Author Slides of Fundamentals of Database Systems”, Ramez Elamsri, Shamkant B Navathe, Pearson, 7th Edition, 2017.

Special Acknowledgement to all CSE Professors who contributed to this endeavor

Dr. Nagasundari S

Department of Computer Science and Engineering



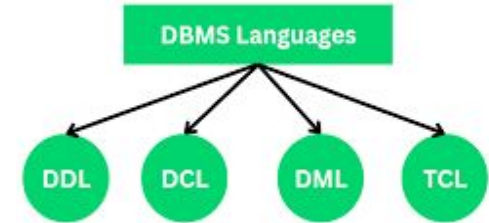
SQL language

- Considered as one of the major reasons for the commercial success of relational databases.

SQL

- SQL actually comes from the word “SEQUEL” which was the original term used in the paper: “SEQUEL TO SQUARE” by Chamberlin and Boyce. IBM could not copyright that term, so they abbreviated to SQL and copyrighted the term SQL.
- Now popularly known as “Structured Query language”.
- The language has features for Data definition, Data Manipulation, Transaction control , Indexing ,Security specification (Grant and Revoke), Active databases, Multi-media , Distributed databases etc.

- **Data Definition Language (DDL)** statements are used to define the database structure or schema.
- **Data Manipulation Language (DML)** statements are used for managing data within schema objects
- **Data Control Language (DCL)** statements are used to control access to the database.
- **Transaction Control Language (TCL)** is used to run the changes made by the DML statement.



SQL schema

- Identified by a **schema name**
- Includes an **authorization identifier** and **descriptors** for each element
- Each statement in SQL ends with a **semicolon**
- Schema elements include tables, constraints, views, domains and other constraints

CREATE SCHEMA statement:

MySQL Syntax:

CREATE SCHEMA Library;

This is same as **CREATE DATABASE Library**; ie schema and database are same in MySql

Equivalent PostgreSQL/Oracle syntax: (for reference)

CREATE SCHEMA Library AUTHORIZATION 'Jsmith';

Basic data types:

Numeric data types

- Integer numbers: INTEGER, INT, and SMALLINT
- Floating-point (real) numbers: FLOAT or REAL, and DOUBLE PRECISION

Character-string data types

- Fixed length: CHAR(n), CHARACTER(n)
- Varying length: VARCHAR(n), CHAR VARYING(n), CHARACTER VARYING(n)

Database Management Systems

Attribute Data Types in SQL



Bit-string data types

- Fixed length: BIT(n)
- Varying length: BIT VARYING(n)

Boolean data type

- Values of TRUE or FALSE

DATE data type

- Ten positions
- Components are YEAR, MONTH, and DAY in the form YYYY-MM-DD
- Multiple mapping functions available in RDBMSs to change date formats

Note: Learn more about MYSQL inbuilt Date Functions [here](#)

Database Management Systems

Attribute Data Types in SQL



Timestamp data type

- Includes the DATE and TIME fields
- Plus a minimum of six positions for decimal fractions of seconds
- Optional WITH TIME ZONE qualifier

INTERVAL data type

- Specifies a relative value that can be used to increment or decrement an absolute value of a date, time, or timestamp

DATE, TIME, Timestamp, INTERVAL data types can be cast or converted to string formats for comparison.

Database Management Systems

Attribute Data Types in SQL Explained



The SQL standard supports a variety of built-in types, including:

- **char(n)**: A fixed-length character string with user-specified length n. The full form, character, can be used instead.
- **varchar(n)**: A variable-length character string with user-specified maximum length n. The full form, character varying is equivalent.
- **int**: An integer (a finite subset of the integers that is machine dependent). The full form, integer, is equivalent.
- **smallint**: A small integer (a machine-dependent subset of the integer type).

Database Management Systems

Attribute Data Types in SQL Explained



- **Numeric(p, d):** A fixed-point number with user-specified precision. The number consists of p digits (plus a sign), and d of the p digits are to the right of the decimal point. Thus, numeric(3,1) allows 44.5 to be stored exactly, but neither 444.5 nor 0.32 can be stored exactly in a field of this type.
- **Real, double precision:** Floating-point and double-precision floating-point numbers with machine-dependent precision.
- **Float(n):** A floating-point number with a precision of at least n digits.
- **Null:** Each type may include a special value called the null value. A null value indicates an absent value that may exist but be unknown or that may not exist at all.
- **ENUM:** Data type in MySQL is a **string object**. It allows us to limit the value chosen from a list of permitted values in the column specification at the time of table creation. It is short for **enumeration**, which means that each column may have one of the specified possible values.
ENUM ('value_1','value_2','value_3')

CHAR

Use **CHAR** when you know the strings will always be of the same size. **Example:** Pincode

Fixed length: storage taken is always equal to the defined length (extra space padded with blanks).

Utilizes the entire defined storage by padding trailing spaces.

Slightly better performance in retrieval due to fixed size.

CHAR can hold **NULL** values if defined as nullable.

VARCHAR

Use **VARCHAR** when the size of strings will vary. **Example:** Name

Variable length: storage taken is only equal to the length of the actual data entered.

Uses only the storage required for actual data (no unnecessary padding).

Slightly lower performance than **CHAR** because of handling variable sizes.

VARCHAR can hold **NULL** values if defined as nullable

The char data type stores fixed-length strings.

Consider, for example, an attribute A of type char(10).

- If we stored a string “Avi” in this attribute, seven spaces are appended to the string to make it 10 characters long.
- In contrast, if attribute B were of type varchar(10), and we stored “Avi” in attribute B, no spaces would be added.

Large-Object Types

- Many current-generation database applications need to store attributes that can be large of the order of many kilobytes/megabytes.
- These data types include as a photograph, or very large (of the order of many megabytes or even gigabytes), such as a high-resolution medical image or video clips.
- SQL therefore provides large-object data types for character data (**clob**) and binary data (**blob**). The letters “lob” in these data types stand for “Large Object.”

Large-Object Types

For example, we may declare attributes in oracle as:

book review **clob**(10KB)

image **blob**(10MB)

movie **blob**(2GB)

- For result tuples containing large objects (multiple megabytes to gigabytes), it is inefficient or impractical to retrieve an entire large object into memory.
- LOBs in the database are stored in a way that optimizes the space and provides efficient access within the database tablespaces.
- Internal LOBs (BLOBs, CLOBs) also provide transactional support (Commit, Rollback, and so on) of the database server.

- BLOB stands for Binary Long Object
- A BLOB is a binary large object that can hold a variable amount of data. You can store files like videos, images, gifs, and audio files. BLOB columns are treated as binary strings that use the binary character.
- Comparisons and sorting are based on raw numeric byte values.
- BLOB cannot have default values.

Types:

- o TINYBLOB: up to 255 bytes
- o BLOB: up to 65,535 bytes (~64 KB)
- o MEDIUMBLOB: up to 16,777,215 bytes (~16 MB)
- o LONGBLOB: up to 4,294,967,295 bytes (~4 GB)

- CLOB stands for Character Long Object
- In MYSQL TEXT acts as the equivalent of CLOB. You can store long textual content that doesn't fit into normal VARCHAR columns.
- Use a proper character set (e.g., utf8mb4) and are stored as character strings
- Sorting/comparison follows the rules of the character set
- For TEXT columns, you must specify a prefix length for index creation. TEXT cannot have default values.

Types:

- o TINYTEXT: up to 255 characters
- o TEXT: up to 65,535 characters
- o MEDIUMTEXT: up to 16,777,215 characters
- o LONGTEXT: up to 4,294,967,295 characters

For a detailed walkthrough on how BLOB and CLOB works, refer to the file below (for your reference - in detail will be dealt in further slides)

[BLOB_CLOB.SQL](#)

Database Management Systems

Single Example covers all data types

```
CREATE TABLE example (  
    id INT(6) UNSIGNED AUTO_INCREMENT PRIMARY KEY,  
    char_column CHAR(50) NOT NULL,  
    varchar_column VARCHAR(255) NOT NULL,  
    tinytext_column TINYTEXT NOT NULL  
    text_column TEXT NOT NULL,  
    mediumtext_column MEDIUMTEXT NOT NULL,  
    longtext_column LONGTEXT NOT NULL,  
    binary_column BINARY(20) NOT NULL,  
    varbinary_column VARBINARY(20) NOT NULL,  
    tinyblob_column TINYBLOB NOT NULL,  
    blob_column BLOB NOT NULL,  
    mediumblob_column MEDIUMBLOB NOT NULL,
```

Database Management Systems

Single Example covers all data types

```
longblob_column LONGBLOB NOT NULL,  
enum_column ENUM('value1', 'value2', 'value3') NOT NULL,  
set_column SET('value1', 'value2', 'value3') NOT NULL,  
bool_column BOOLEAN NOT NULL DEFAULT 0,  
date_column DATE NOT NULL,  
datetime_column DATETIME NOT NULL,  
timestamp_column TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP ON  
UPDATE CURRENT_TIMESTAMP,  
time_column TIME NOT NULL,  
year_column YEAR(4) NOT NULL,  
float_column FLOAT(10,6) NOT NULL DEFAULT '0.000000',  
double_column DOUBLE(10,6) NOT NULL DEFAULT '0.000000'  
);
```

Terminology:

- **Table**, **row**, and **column** used for relational model terms relation, tuple, and attribute respectively.

CREATE statement

- Main SQL command for data definition
- The CREATE TABLE command is used to create a new relation by giving it a name and specifying its attributes and initial constraints.

- The SQL DDL provides commands for defining table schemas, deleting tables, and modifying table schemas.
- DDL includes commands such as CREATE, ALTER, and DROP.
- DDL is used to CREATE, ALTER, OR DROP the database objects (Table, Views, Users).
- **CREATE** - to create objects in the database
- **ALTER** - alters the structure of the database
- **DROP** - delete objects from the database
- **TRUNCATE** - remove all records from a table, including all spaces allocated for the records are removed
- **COMMENT** - add comments to the data dictionary
- **RENAME** - rename an object

Types of Database Languages: DDL

CREATE TABLE

Syntax:

```
CREATE TABLE  
table_name(  
    Col_name1 datatype(),  
    Col_name2 datatype(),...  
    Col_namen datatype(),  
);
```

```
CREATE TABLE `dept_locations` (  
    `Dnumber` int(11) NOT NULL,  
    `Dlocation` varchar(15) NOT NULL  
);
```

Create table from existing table:

```
CREATE TABLE new_table_name AS SELECT *  
FROM existing_table;  
  
CREATE TABLE new_table_name LIKE  
existing_table;
```

```
CREATE TABLE `project` (  
    `Pname` varchar(15) NOT NULL,  
    `Pnumber` int(11) NOT NULL,  
    `Plocation` varchar(15) DEFAULT NULL,  
    `Dnum` int(11) NOT NULL  
);
```

Types of Database Languages: DDL

Difference between CREATE TABLE new_table_name **AS SELECT** * FROM existing_table;
and CREATE TABLE new_table_name **LIKE** existing_table;

Feature	as select	like
Copies structure	Yes	Yes
Copies data	Yes	No
Copies constraints/indexes	No	Yes
Copies AUTO_INCREMENT	No	Yes
Flexibility in copying data	Yes can use where and join	No

Types of Database Languages: DDL

ALTER TABLE

1) ADD

Syntax:

ALTER TABLE table_name

ADD Col_name datatype()...;

2) MODIFY

Syntax:

ALTER TABLE table_name

MODIFY (fieldname datatype()...);

[Learn more examples](#)

3) RENAME

Syntax:

ALTER TABLE table_name

RENAME COLUMN (Old_fieldname TO
New_fieldname...);

4) DROP

Syntax:

ALTER TABLE table_name **DROP** column_name;

Types of Database Languages: DDL

C) DESCRIBE TABLE

Syntax:

```
DESCRIBE table_name;
```

//lists basic description.

```
SHOW CREATE TABLE table_name;
```

// lists complete information of table including constraints,index etc..

D) DROP TABLE

Syntax: DROP TABLE table_name;

// Complete table structure will be dropped

E) RENAME

Syntax:

```
RENAME TABLE table_name to new  
table_name
```

F) TRUNCATE

Syntax:

```
TRUNCATE TABLE table_name; // delete  
complete data from an existing table. Table  
Structure remains
```

Types of Database Languages: DDL

Feature	DROP	TRUNCATE
Removes data	Yes	Yes
Removes table structure	Yes (table is deleted)	No (structure remains)
Auto_Increment reset	N/A (table removed)	Resets to 1 (in MySQL)
Constraints & Indexes	Deleted along with table	Preserved (only rows deleted)
Rollback (MySQL)	Cannot be rolled back	Cannot be rolled back (rollback dealt in further slides)
Use case	When table is no longer needed	When you want to empty the table but keep its structure

EMPLOYEE

Fname	Minit	Lname	<u>Ssn</u>	Bdate	Address	Sex	Salary	Super_ssn	Dno
-------	-------	-------	------------	-------	---------	-----	--------	-----------	-----

DEPARTMENT

Dname	<u>Dnumber</u>	Mgr_ssn	Mgr_start_date
-------	----------------	---------	----------------

DEPT_LOCATIONS

<u>Dnumber</u>	<u>Dlocation</u>
----------------	------------------

PROJECT

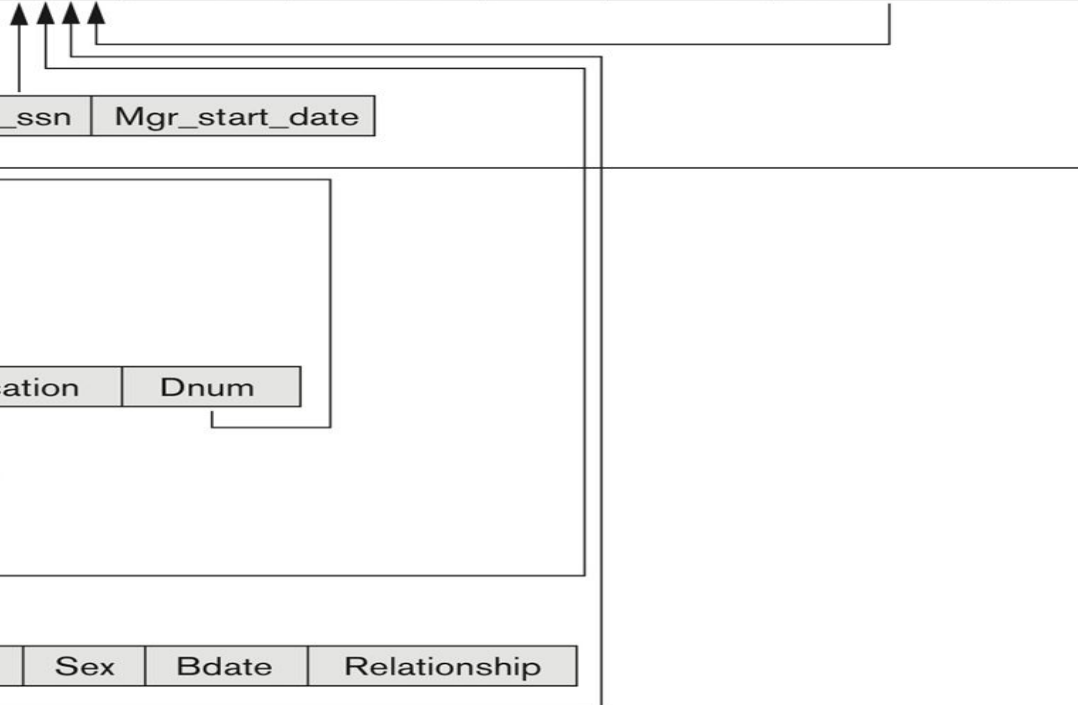
Pname	<u>Pnumber</u>	Plocation	Dnum
-------	----------------	-----------	------

WORKS_ON

<u>Essn</u>	<u>Pno</u>	Hours
-------------	------------	-------

DEPENDENT

<u>Essn</u>	<u>Dependent_name</u>	Sex	Bdate	Relationship
-------------	-----------------------	-----	-------	--------------



- CREATE TABLE `department` (`Dname` varchar(15) NOT NULL, `Dnumber` int(11) NOT NULL, `Mgr\_ssn` char(9) NOT NULL, `Mgr\_start\_date` date DEFAULT NULL);
- CREATE TABLE `dependent` (`Essn` char(9) NOT NULL, `Dependent\_name` varchar(15) NOT NULL, `Gender` char(1) DEFAULT NULL, `Bdate` date DEFAULT NULL, `Relationship` varchar(8) DEFAULT NULL);
- CREATE TABLE `dept\_locations` (`Dnumber` int(11) NOT NULL, `Dlocation` varchar(15) NOT NULL);

SQL CREATE TABLE data definition statements for defining the COMPANY schema

- CREATE TABLE `employee` (`Fname` varchar(15) NOT NULL, `Minit` char(1) DEFAULT NULL, `Lname` varchar(15) NOT NULL, `Ssn` char(9) NOT NULL, `Bdate` date DEFAULT NULL, `Address` varchar(30) DEFAULT NULL, `Gender` char(1) DEFAULT NULL, `Salary` decimal(10,2) DEFAULT NULL, `Super\_ssn` char(9) DEFAULT NULL, `Dno` int(11) NOT NULL);
- CREATE TABLE `project` (`Pname` varchar(15) NOT NULL, `Pnumber` int(11) NOT NULL, `Plocation` varchar(15) DEFAULT NULL, `Dnum` int(11) NOT NULL);
- CREATE TABLE `works\_on` (`Essn` char(9) NOT NULL, `Pno` int(11) NOT NULL, `Hours` decimal(3,1) DEFAULT NULL);

- Comments can be added to the table/column
- The comments serve as metadata, providing descriptions for database objects.
- SHOW CREATE TABLE table_name displays the comments for the table/column.

Example:

Comment at the table level:

```
CREATE TABLE users (  
    id INT PRIMARY KEY,  
    name VARCHAR(255)  
) COMMENT 'This table stores information about users.';
```

Comment at the column level:

```
CREATE TABLE products (  
    product_id INT PRIMARY KEY COMMENT 'Unique identifier for the product',  
    product_name VARCHAR(255) COMMENT 'Name of the product',  
    price DECIMAL(10, 2) COMMENT 'Price of the product'  
);
```

Database Management Systems

One possible database state for the COMPANY relational database schema



EMPLOYEE

Fname	Minit	Lname	Ssn	Bdate	Address	Sex	Salary	Super_ssn	Dno
John	B	Smith	123456789	1965-01-09	731 Fondren, Houston, TX	M	30000	333445555	5
Franklin	T	Wong	333445555	1955-12-08	638 Voss, Houston, TX	M	40000	888665555	5
Alicia	J	Zelaya	999887777	1968-01-19	3321 Castle, Spring, TX	F	25000	987654321	4
Jennifer	S	Wallace	987654321	1941-06-20	291 Berry, Bellaire, TX	F	43000	888665555	4
Ramesh	K	Narayan	666884444	1962-09-15	975 Fire Oak, Humble, TX	M	38000	333445555	5
Joyce	A	English	453453453	1972-07-31	5631 Rice, Houston, TX	F	25000	333445555	5
Ahmad	V	Jabbar	987987987	1969-03-29	980 Dallas, Houston, TX	M	25000	987654321	4
James	E	Borg	888665555	1937-11-10	450 Stone, Houston, TX	M	55000	NULL	1

DEPARTMENT

Dname	Dnumber	Mgr_ssn	Mgr_start_date
Research	5	333445555	1988-05-22
Administration	4	987654321	1995-01-01
Headquarters	1	888665555	1981-06-19

DEPT_LOCATIONS

Dnumber	Dlocation
1	Houston
4	Stafford
5	Bellaire
5	Sugarland
5	Houston

Database Management Systems

One possible database state for the COMPANY relational database schema



WORKS_ON

<u>Essn</u>	<u>Pno</u>	Hours
123456789	1	32.5
123456789	2	7.5
666884444	3	40.0
453453453	1	20.0
453453453	2	20.0
333445555	2	10.0
333445555	3	10.0
333445555	10	10.0
333445555	20	10.0
999887777	30	30.0
999887777	10	10.0
987987987	10	35.0
987987987	30	5.0
987654321	30	20.0
987654321	20	15.0
888665555	20	NULL

PROJECT

<u>Pname</u>	<u>Pnumber</u>	Plocation	Dnum
ProductX	1	Bellaire	5
ProductY	2	Sugarland	5
ProductZ	3	Houston	5
Computerization	10	Stafford	4
Reorganization	20	Houston	1
Newbenefits	30	Stafford	4

DEPENDENT

<u>Essn</u>	<u>Dependent_name</u>	Sex	Bdate	Relationship
333445555	Alice	F	1986-04-05	Daughter
333445555	Theodore	M	1983-10-25	Son
333445555	Joy	F	1958-05-03	Spouse
987654321	Abner	M	1942-02-28	Spouse
123456789	Michael	M	1988-01-04	Son
123456789	Alice	F	1988-12-30	Daughter
123456789	Elizabeth	F	1967-05-05	Spouse

1. What is the difference between CHAR(n) and VARCHAR(n)?

- A) CHAR(n) stores variable-length strings, VARCHAR(n) stores fixed-length strings
- B) CHAR(n) stores fixed-length strings, VARCHAR(n) stores variable-length strings
- C) Both store fixed-length strings
- D) Both store variable-length strings

2. What is the format of the DATE data type in SQL?

- A) DD-MM-YYYY
- B) MM-DD-YYYY
- C) YYYY-MM-DD
- D) Any format can be used

3. What is a key difference between BLOB and CLOB data types?

- A) BLOB is used for large binary objects, CLOB is used for large character data
- B) Both store large binary objects
- C) Both store large character objects
- D) CLOB is used for numbers only

4. What is the purpose of the ENUM data type in MySQL?

- A) Storing fixed-length character data
- B) Storing numeric data
- C) Defining a list of possible values for a column
- D) Storing date and time values

Answers:

1. B
2. C
3. A
4. C



THANK YOU

Anchor: Dr. Nagasundari S, Dept of CSE

nagasundaris@pes.edu